

YAZILIM MÜHENDİSLİĞİ

HAZIRLAYAN VE SUNAN

Doç. Dr. Asım Sinan YÜKSEL



asimyuksel@sdu.edu.tr

3. Ders



SÜLEYMAN
DEMİREL
UNİVERSİTESİ



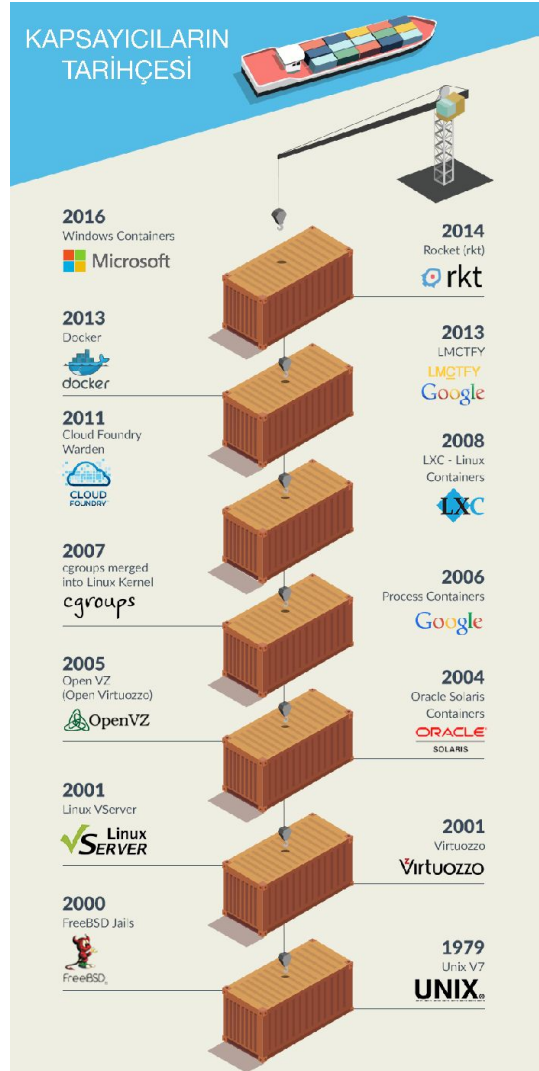
BİLGİSAYAR
MÜHENDİSLİĞİ



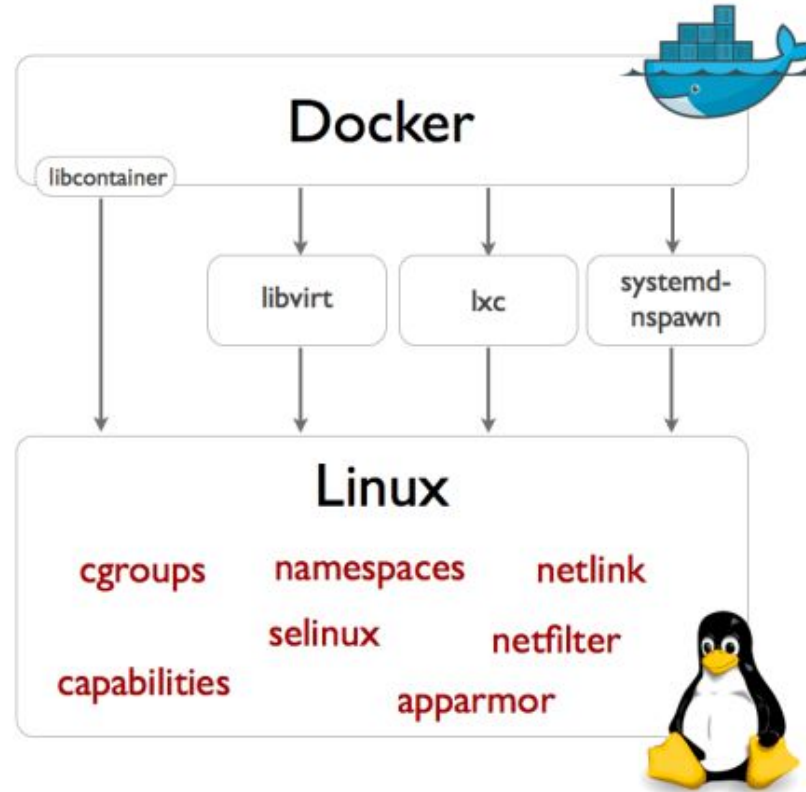
Docker

- Servis olarak platform (PaaS, Platform as a Service) sunan dotCloud isimli bir firma özelleştirilmiş bir konteynır kullanıyordu.
- Temelinde OpenZV isimli Linux tabanlı bir sanallaştırma teknolojisi vardı.
- İlk başlarda (2008) tek bir Python skriptinden oluşuyordu. 2012 yılında yaklaşık 10 Python bileşini, 100 mikroservisi olan bir yapıya dönüştü.
- 2012 yılı sonunda firma bu konteynırı iyileştirdi ve bu konteynıra kod adı olarak Docker isimi verildi.
- 2013'te kitlelere tanıtıldı ve açık kaynak lisanslı olarak piyasaya sürüldü.
- Firma ismini de Docker olarak değiştirdi.
- Bu tarz yazılımların adı Türkçe olarak kapsayıcı kelimesi ile ifade edilmektedir.
- Kapsayıcıların temel amacı uygulamaların yazılım kapsayıcıları içinde otomatik olarak dağıtımını sağlamaktır.
- Bu hedefe Linux üzerinde, sistem düzeyinde sanallaştırma ve ekstra soyutlama katmanları sunularak ulaşılır.
- Bir yazılımın çalışması için sunucaya yükleyebileceğiniz ne varsa bunların hepsini bir dosya sistemi içinde yazılım olarak size sunabilme imkanına sahiptir.

Kapsayıcıların (Containers) Geçmişi



Docker'ın Linux ile Bağlantısı



İlk çıktığında `libvirt`, `LXC`, ve `systemd-nspawn` kütüphaneleri ile Linux kerneline erişirken şimdi geliştirilen `libcontainer` kütüphanesi ile Linux kernele erişmektedir.

Docker

- **libcontainer:** Şu anki adı **runc**'dir. OCI (Open Container Initiative) standartlarına uygun bir şekilde kapsayıcının çalışması ve kopyalarının üretilmesi için gerekli komut satırı araçlarını içerir.
- **libvirt:** Platform sanallaştırmasını yönetmek için açık kaynaklı bir API, arka plan programı ve yönetim aracıdır.
- **systemd-nspawn:** chroot tarzı bir komuttur. chroot komutunun steroid kullanan versiyonu diyebiliriz. Bir kapsayıcı içinde komut ve işletim sistemi çalıştırmanızı sağlar.
- **cgroups:** bir süreç (process) grubunun kaynak kullanımını sınırlayan, izole eden bir Linux çekirdek özelliğidir.
- **namespaces:** Linux çekirdek kaynaklarını bölümlere ayırır. Bir işlem kümesi bir kaynak kümesini görürken başka bir işlem kümesi farklı bir kaynak kümesi görür. Konteynerlerin taşınabilir kalması ve ana sistemin geri kalanını etkilememesi için ihtiyaç duyduğu yalıtımı sağlar.
- **Netlink:** Kullanıcı alanındaki iş parçacıkları ile kernel arasında bilgi akışını sağlar.
- **SELinux:** Linux çekirdeği güvenlik modülüdür.
- **Netfilter:** Linux çekirdeği tarafından sağlanan, ağ ile ilgili çeşitli işlemlerin özelleştirilmiş işlemler biçiminde uygulanmasına izin veren bir çerçevedir.
- **AppArmor:** Sistem yöneticisinin programların yeteneklerini program bazlı profillerle kısıtlamasını sağlayan bir Linux çekirdek güvenlik modülüdür.
- **capabilities:** İş parçacıklarına root düzeyinde yetkiler, yetki altkümeleri atamanızı sağlayan bir özelliktir.

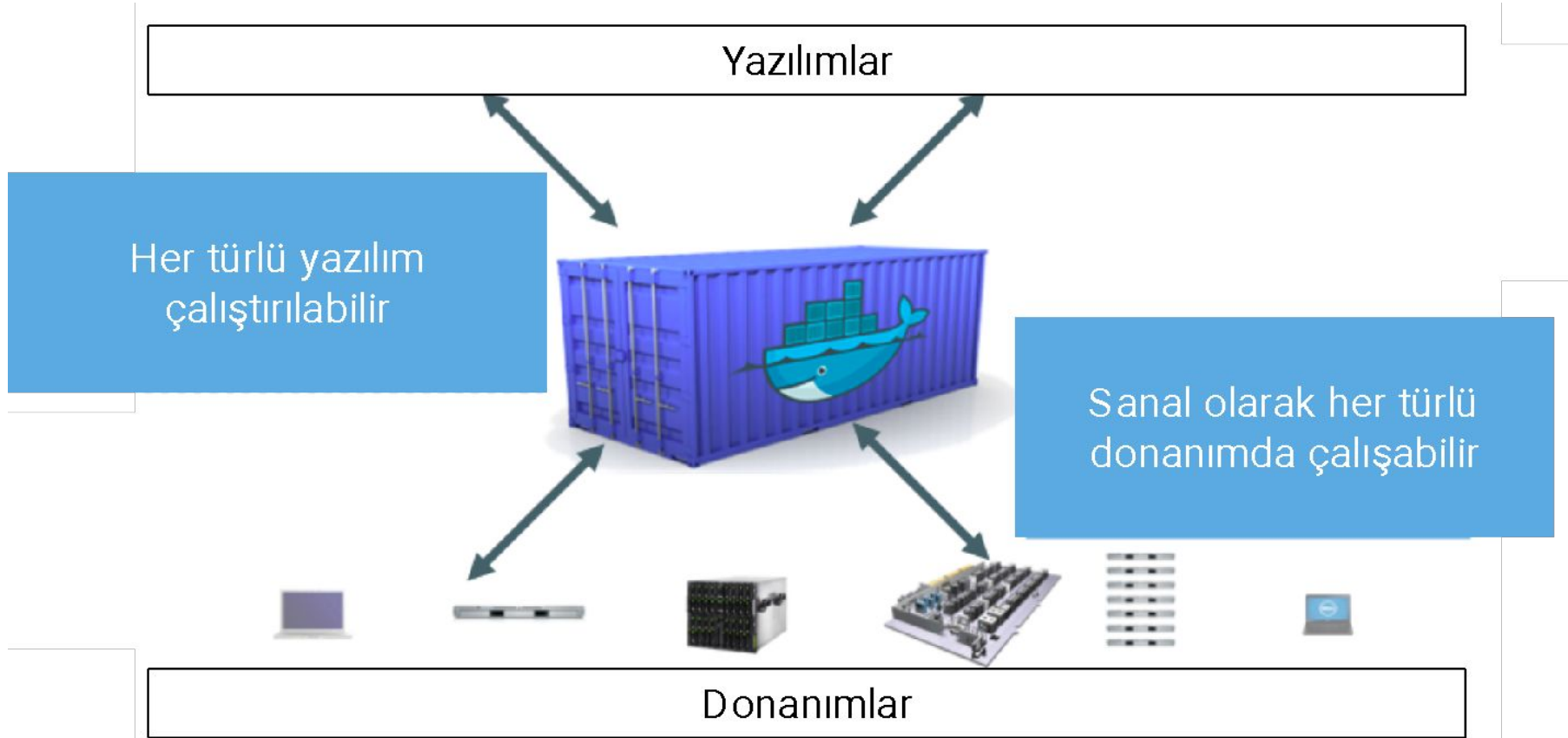
Docker Benzetimi

Esin Kaynağı: Deniz Taşımacılığı



- Kargoların % 90'ı nakliye konteynırları ile taşınır.
- Kargoların yüklenme ve indirilme maliyetleri, süreleri azalmıştır.
- Hırsızlık, kargonun zarar görmesi gibi problemleri azaltmıştır.
- Taşımacılık maliyetlerini % 25'den % 3'e düşürmüştür.
- Yılda ortalama 5000 gemi 200 Milyon konteynır ile çalışmaktadır.

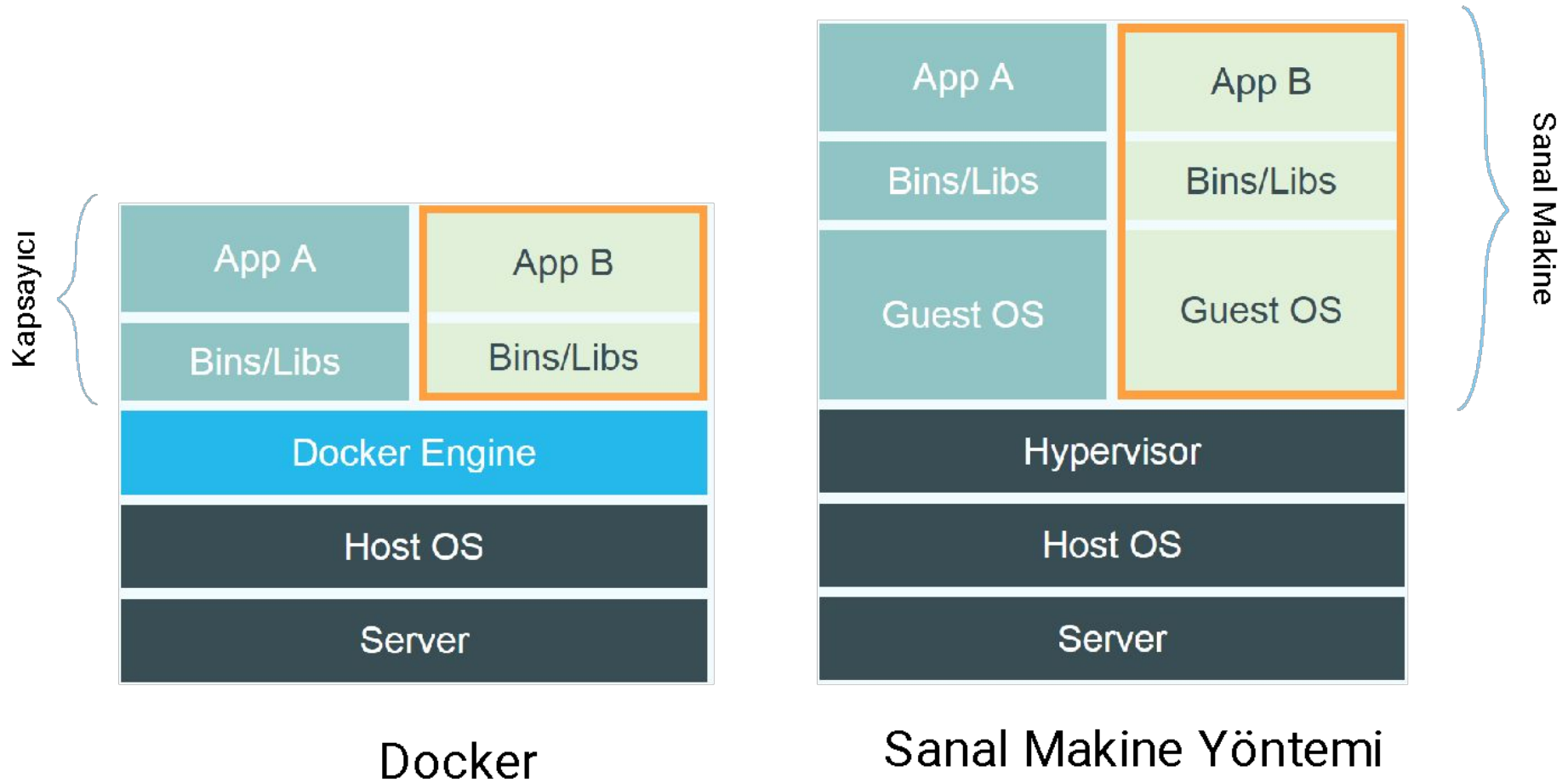
Docker



Docker

- Go dili ile yeniden yazılmıştır.
- Docker sadece işletim sistemini sanallaştırır. Tüm sistemi sanallaştırmaz.
- Sunucu kernel'i üzerinde çalışır. Bazı kütüphaneleri ve derlenmiş uygulamaları (binaries) paylaşırlar. Bunlar sadece okuma yetkisine sahiptir.
- Sistem kaynaklarını kullandığı için boyut olarak daha küçüktürler (lightweight).
- Docker öncesi:
 - Uygulamalar dep, rpm, gem, jar paketleri halinde dağıtılırdı.
 - Diğer paketlere bağımlılık üst düzeydeydi.
 - Sadece "Kendi makinemde çalışıyor, sadece şu versiyonla çalışıyor, şu kütüphaneyi yüklersen çalışır" mantığı vardı.
 - Yeniden dağıtma işlemi her zaman en baştan yapılırdı.
- Docker sonrası:
 - Kapsayıcı imajlar tüm bağımlılıklarıyla üretilmeye başlandı.
 - İmajlar katmanlara ayrılabilirdi.
 - Sadece değişen katmanlar dağıtılabildi.
 - Disk, bellek ve ağ kullanımından kazanç sağlandı.

Docker vs. Sanal Makine



Docker vs. Sanal Makine

Docker vs. Sanal Makine (VM)

VM	Kapsayıcı
Büyük Boyutlu	Küçük Boyutlu
Kısıtlı Performans	Yüksek Performans
Her VM kendi OS'ini çalıştırır	Her kapsayıcı sunucu OS'unu paylaşır
Donanım düzeyinde sanallaştırma	OS sanallaştırması
Dakikalar içinde açılır	Milisaniyeler içinde açılır
İhtiyacı olan hafızayı önceden ayırır	Daha az bellek gereksinimi vardır
Tamamen izole edilmiş, daha güvenli	İşlem düzeyinde izolasyon, daha az güvenli

Docker vs. Sanal Makine (VM, Virtual Machine)

- Uygulamaların geliştirme ortamından üretim ortamına aktarılması için disk imajı oluşturulması gerekir.
- Hızlı imaj oluşturma hem hızlı test hem de daha sık dağıtım imkanı sağlar.
- VM tüm işletim sistemi imajını içerirken, Docker sadece fark imajını (uygulama katmanını) içerir.
- VM:
 - Bağımlılık cehennemi: Uygulamaların %50'den az bir kısmı derlenebilir ve yüklenebilir.
 - Bilimsel hesaplamalar için kullanımı çok zor.
 - Dokümantasyon sıkıntısı: Sanal makinelerin nasıl kurulacağı dair az ya da eksik belge olması.
 - Yeniden kullanımı, genişlemesi, paket yönetici oluşturmaları çok zor.
 - Bulut ortamlarına ya da diğer ortamlara taşınabilirliği zorlaştırır.

Docker vs. VM Performans Karşılaştırması

Zaman (saniye)	VM	Docker
MySQL	236	129
NodeJS	304	49

Docker: 2x-6x daha hızlı

Boyut	VM	Docker
MySQL	1.68GB	112KB
NodeJS	2.05GB	72KB

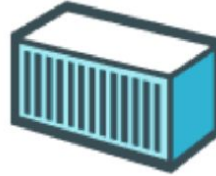
Docker: 2x-6x daha küçük

Docker'ın Ana Hedefi



Geliştir

Docker kapsayıcısı ile herhangi bir dilde uygulama geliştirin



Dağıt

Hiç birşeyi bozmadan uygulama ve bağımlılıkları kapsülleyerek (Dockerize) istediğiniz takıma, buluta dağıtın

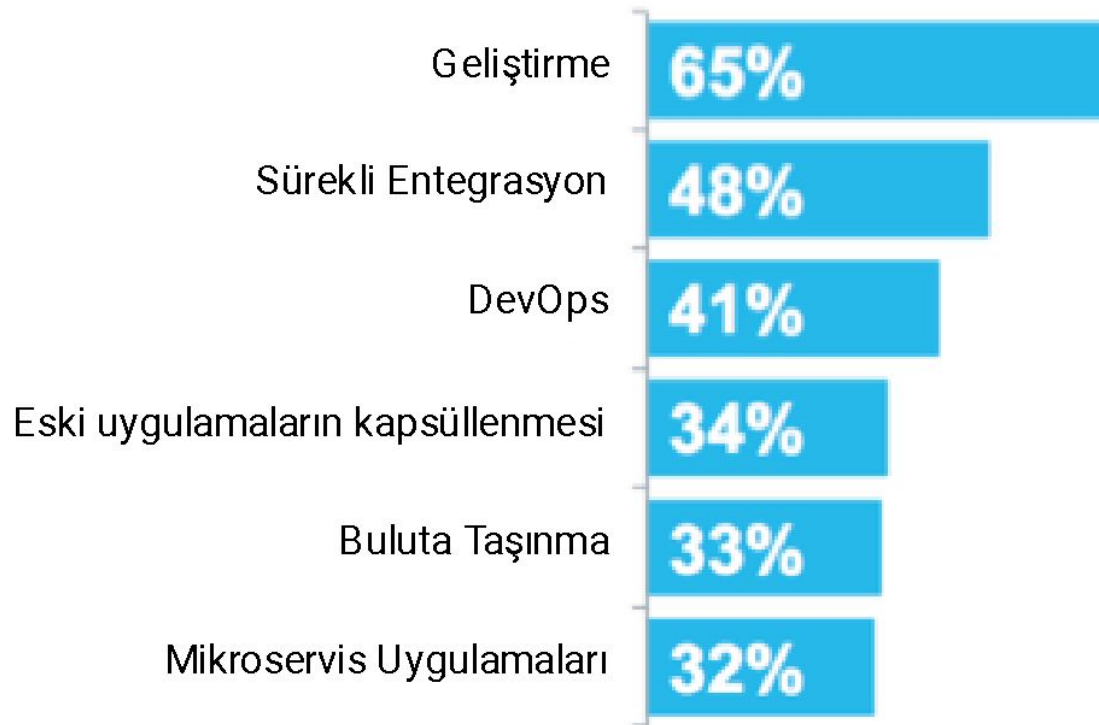


Çalıştır

Veri merkezlerinde, bulutta her ortamda çalıştırın, genişleyin.

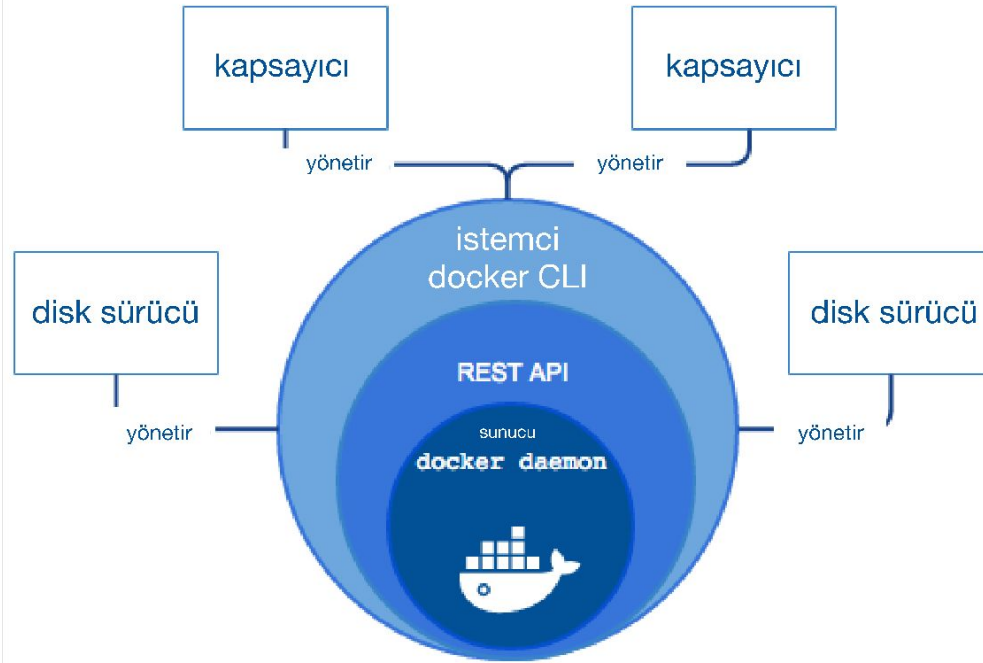
Docker Kullanım Senaryoları

Docker Kapsayıcılar Nasıl Kullanılıyor?

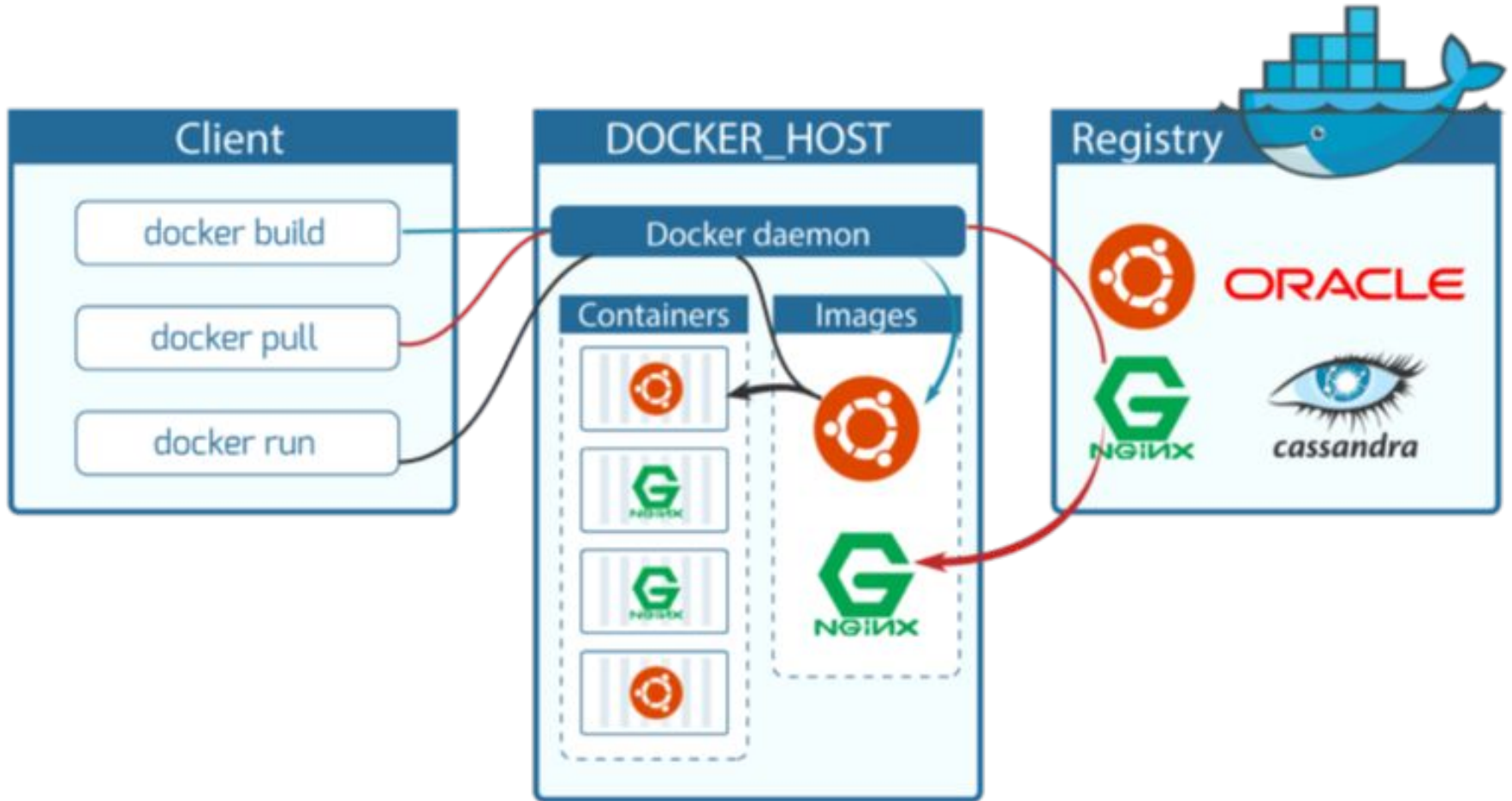


Docker Engine

- Docker Engine 3 ana bileşenden oluşan bir istemci-sunucu uygulamasıdır.
- **Sunucu (Server):** Arka planda sürekli çalışan daemon process olarak adlandırılan bir uygulama (dockerd komutu). Daemon, imajları, disk sürücülerini, ağı yönetir.
- **REST API:** Programların daemon ile konuşabilmelerini, daemon'a komut verebilmelerini sağlar.
- **CLI:** İstemci tarafından kullanılan komut satırı arayüzü (docker komutu). REST API'yi kullanarak daemona ulaşabilir.



Docker Mimarisi



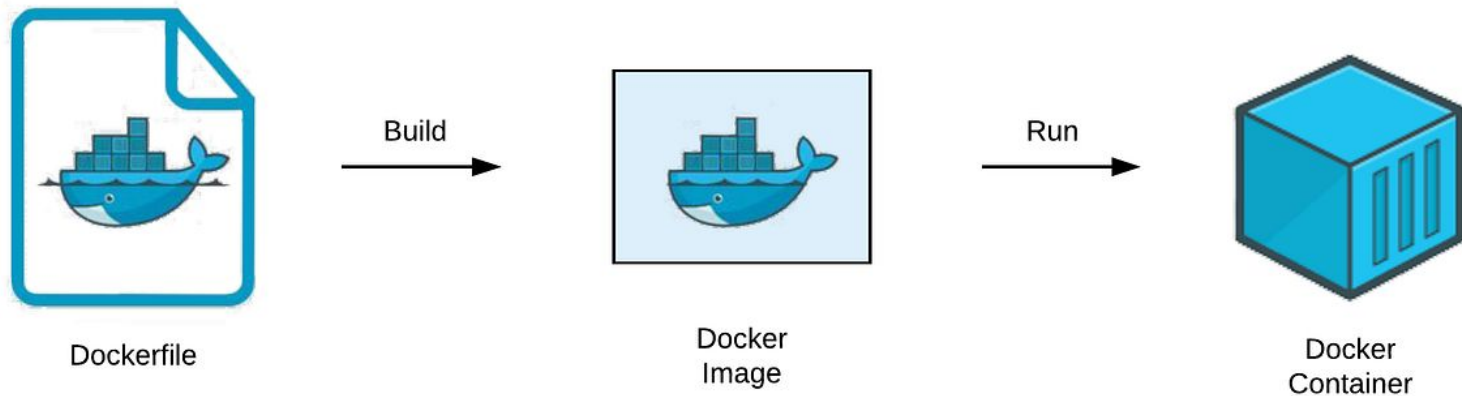
Docker Mimarisi

- Docker istemci-sunucu mimarisini kullanır.
- Docker istemcisi tüm işi yapan Docker daemon ile konuşur.
- İstemci ile daemon aynı sistem üzerinde çalışabilir. Ya da istemci uzaktaki daemona bağlanabilir.
- İstemci ve daemon REST API aracılığı ile Unix soket yapısı ya da ağ arayüzleri üzerinden haberleşir.
- İstemci **docker run** gibi bir komut çalıştırdığında, istemci bu komutu **dockerd**'ye göndermiş olur.
- **docker** komutu Docker API'sini kullanır. İstemci birden fazla daemon ile haberleşebilir.
- **Docker Registry (Kayıtçı):** Docker imajlarını tutar. [Docker Hub](#) herkese açık, herkesin kullanabileceği bir kayıtçıdır (imaj deposu da diyebiliriz).
- Aynı zamanda kendi özel kayıtçılarınızı da oluşturabilirsiniz.
- **docker pull** ya da **docker run** komutlarını çalıştırdığınızda gerekli imajlar ayarladığınız kayıtçıdan indirilir ve çalıştırılır.
- **docker push** komutunu kullandığınızda imajınız ayarladığınız kayıtçıya gönderilir.

Docker Mimarisi

- **Docker Nesneleri (Objects):** Docker'ı kullandığınızda imajlar, kapsayıcılar, ağlar, disk sürücüler, eklentiler ya da başka nesneler üretip kullanabilirsiniz. Bunların hepsi birer Docker nesnesidir.
- **İmajlar (Images):**
 - Docker kapsayıcıları oluşturmanızı sağlayan sadece okuma iznine sahip olduğunuz ve içinde talimatlar içeren bir şablondur.
 - İmajlar genelde başka bir imajı temel alır. Örneğin Ubuntu tabanlı bir imaj oluşturabilir, Apache sunucusunu ve uygulamanızı, uygulamanızın çalışması için gerekli konfigürasyonları da bu imaja ekleyebilirsiniz.
 - Kendi imajınızı oluşturabilir ya da açık erişimli kayıtçılardan imaj indirip üzerinde değişiklikler yapabilirsiniz.
- **Dockerfile:** Bu dosya aracılığı ile kendi imajınızı oluşturup çalıştırabilirsiniz. Bu dosyanın içinde imajın nasıl oluşturulacağı ve çalıştırılacağı ile ilgili adım adım talimatlar yer alır. Dosyadaki her talimat imajda bir katman oluşturur. Dockerfile'ı değiştirip imajınızı yeniden oluşturduğunuzda sadece değişen katmanlar yeniden oluşturulur.

Docker Mimarisi



Dockerfile Örnek

```
# syntax=docker/dockerfile:1

FROM node:18-alpine
WORKDIR /app
COPY . .
RUN yarn install --production
CMD ["node", "src/index.js"]
EXPOSE 3000
```

Dockerfile Örnek

```
FROM microsoft/dotnet:sdk AS build-env
WORKDIR /Docker

# Copy csproj and restore as distinct layers
COPY *.csproj ./
RUN dotnet restore

# Copy everything else and build
COPY . ./
RUN dotnet publish -c Release -o out

# Build runtime image
FROM microsoft/dotnet:aspnetcore-runtime
WORKDIR /Docker
COPY --from=build-env /Docker/out .
ENTRYPOINT ["dotnet", "aspnetapp.dll"]
```

Dockerfile Komutlar

- **FROM:** İmaj oluşturma adımını başlatarak kullanılacak olan temel imajı (base image) ayarlar. Bu satırdan sonra yazılan komutlar bu imaj tarafından kullanılır. Bu yüzden Dockerfile'daki ilk komut FROM olmalıdır.
- **WORKDIR:** Kapsayıcının çalışma klasörünü ayarlar. Container içerisinde belirtilen bu yola geçiş yapılır. Eğer belirtilen yol mevcut değilse öncelikle oluşturulur ve sonrasında bu yola geçiş yapılır.
- **COPY:** İstemcide yer alan dosyaları, klasörleri kapsayıcıya kopyalar.
- **ADD:** COPY ile aynı işi yapar. Ek olarak URL kopyalamayı ve tar dosyalarının içeriğinin çıkarılmasını aktifleştirir.
- **RUN:** Komut çalıştırır ve dönen sonuçları imajda yeni bir katman olarak kaydeder. Genellikle yazılım paketlerini çalıştırmak için kullanılır.
- **LABEL:** Kapsayıcı hakkında anahtar-değer ilişki şeklinde metada tutar.
- **MAINTAINER:** İmajdan sorumlu kişinin bilgilerini eklemeyi sağlar.

Dockerfile Komutlar

- **CMD:** Kapsayıcının çalışmasıyla birlikte hangi komutun çalıştırılacağını bilgilendirir. Dockerfile içerisinde sadece bir tane CMD komutu kullanılabilir. Temel amaç kapsayıcı çalıştığında içindeki uygulamayı çalıştırmak. Birden fazla kullanılıyorsa da en son kullanılan CMD komutu geçerli olacaktır.
- **EXPOSE:** İmajda hangi portların açılacağını belirtir.
- **VOLUME:** Belirtilen yola disk sürücüsü bağlar.
- **ENV:** Anahtar-Değer ilişkisi biçiminde ortam değişkenlerini ayarlamanızı sağlar.
- **USER:** İmaj çalışırken kullanılacak kullanıcı adını belirler.
- **ENTRYPOINT:** CMD'ye benzer. Komuta ek parametre eklemenizi sağlar. Kapsayıcı çalıştırdığınızda CMD'ye verdiğiniz parametreyi değiştiremezken ENTRYPOINT kullandığında kapsayıcı varsayılan parametrenin üzerine yazıp yeni komutu çalıştırabilir.
- Kapsayıcı içerisine eklenmesini istemediğimiz dosyalar varsa **.dockerignore** isimli dosya içerisinde belirtilebilirsiniz.

Docker Mimarisi

- **Kapsayıcılar (Containers):**

- İmajın çalıştırılabilir örneğidir.
- Docker API ya da CLI kullanarak kapsayıcıları başlatabilir (start), durdurabilir (stop), taşıyabilir (move), ya da silebilirsiniz (delete).
- Kapsayıcıları bir ya da daha fazla ağa bağlayabilirsiniz.
- Disk bağlayabilirsiniz.
- Varsayılan olarak diğer kapsayıcılardan ve sunucudan iyi izole edilmiştir. İzolasyonun ne düzeyde olacağını da kontrol edebilirsiniz.

- ***docker run -i -t ubuntu /bin/bash*** gibi bir komut çalıştırdığınızda:

- Eğer ubuntu imajı bilgisayarınızda yoksa kayıtçıdan çekilir. ***docker pull*** ile kendiniz de çekebilirsiniz.
- Docker yeni bir kapsayıcı oluşturur.
- Kapsayıcıya okuma-yazma yetkili dosya sistemi ayırır. Kendi sistemindeki dosyalara erişmek, dosyaları değiştirmek için izin sağlar.
- Varsayılan ağa bağlanmak için gerekli ağ arayüzünü oluşturur. IP atama işlemini yapar.
- Docker kapsayıcıyı başlatır ve -i ve -t bayraklarından dolayı interaktif olarak ve terminale bağlayarak /bin/bash" i çalıştırır.
- ***exit*** komutunu çalıştırırsanız **bin/bash**'ten çıkılır. Kapsayıcı durur ancak kaldırılmaz.

Docker Mimarisi

```
[sinanyuksel@Sinans-MacBook-Pro ~ % docker run -i -t ubuntu /bin/bash
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
2ab09b027e7f: Pull complete
Digest: sha256:67211c14fa74f070d27cc59d69a7fa9aeff8e28ea118ef3babbc295a0428a6d21
Status: Downloaded newer image for ubuntu:latest
[root@09ac66ed1085:/# ls
bin    dev    home  lib32  libx32  mnt    proc  run    srv    tmp    var
boot  etc    lib   lib64  media   opt    root  sbin   sys    usr
[root@09ac66ed1085:/# exit
exit
sinanyuksel@Sinans-MacBook-Pro ~ %
```

Docker Mimarisi

<

ubuntu:latest

IN USE

08d22c0ceb15

CREATED

26 days ago

SIZE

77.81 MB

Recommended fixes

▼

Run

▼

[Docker Scout free early access](#)

Image hierarchy

ALL

ubuntu:latest

Layers (6)

0	ARG RELEASE	0 B	
1	ARG LAUNCHPAD_BUILD_ARCH	0 B	
2	LABEL org.opencontainers.im...	0 B	
3	LABEL org.opencontainers.im...	0 B	
4	ADD file:c8ef6447752cab254...	77.81 MB	
5	CMD ["/bin/bash"]	0 B	

Images (1)

Vulnerabilities (7)

Packages (143)

[Give feedback](#)

ubuntu:latest

0 H

0 M

7 L

Packages: 143

Docker Mimarisi

- **Disk Sürücüleri (Volumes):**

- Docker tarafından oluşturulan ve Docker kapsayıcıları tarafından kullanılan veriler disk sürücülerinde tutulur.
- Docker API ve CLI ile yönetilir.
- Hem Windows hem de Linux kapsayıcılar ile çalışabilir.
- Sürücünün içeriği kapsayıcı yaşam döngüsünün dışında yer alır. Dolayısıyla kapsayıcının boyutunda değişikliğe neden olmaz.
- -v ya da --mount bayraklarını kullanarak kapsayıcınızı bir disk sürücüsü ile başlatabilirsiniz.

- **Ağlar (Networks):**

- **Bridge:** Kapsayıcı için varsayılan ağ sürücüsüdür. Uygulamanız kapsayıcı içinde çalışıyorsa ya da birden fazla kapsayıcıda çalışıp tek docker sunucusu ile haberleşecekse bu seçeneği kullanabilirsiniz.
- **Host:** Docker sunucusu ile docker istemcisi arasındaki ağ izolasyonunu kaldırır.
- **Overlay:** Kapsayıcılar farklı sunucularda çalışıyorsa bu seçenek kullanılır.
- **None:** Ağ yok.
- **macvlan:** Kapsayıcıların fiziksel bir aygıtımsı gibi MAC adresi almalarını sağlar.

Docker Komutları

- **docker create --name [KAPSAYICIADI] [IMAJ]:** Kapsayıcıyı başlatmadan belirtilen imajdan bir kapsayıcı oluşturur.
 - `docker create --name benimkapsayicim alpine`
 - Alpine linux imajından benim kapsayicim adıyla bir kapsayıcı oluşturur.
- **docker rename [ESKI_AD] [YENI_AD]:** Kapsayıcıyı yeniden adlandırır.
 - `docker rename benimkapsayicim seninkapsayicin`
- **docker run [IMAJ]:** İmajdan kapsayıcı oluşturur ve başlatır.
 - `docker run -i -t --name benimkapsayicim alpine`
- **docker run --rm [IMAJ]:** Kapsayıcı oluşturur ve başlatır ve kapsayıcı durduktan sonra kaldırır.
 - `docker run -t -i --rm ubuntu /bin/bash`

Docker Komutları

- **docker run -td [IMAJ]:** Kapsayıcıyı oluşturur, çalıştırır ve sürekli çalışmasını sağlar.
 - docker run -td alpine
- **docker run -it [IMAJ]:** Kapsayıcıyı oluşturur, çalıştırır ve içinde komut çalıştırmayı sağlar.
 - docker run --name test -it debian
- **docker run -it --rm [IMAJ]:** Kapsayıcıyı oluşturur, çalıştırır ve içinde komut çalıştırmayı sağlar, çıkınca kapsayıcıyı kaldırır.
- **docker rm [KAPSAYICI]:** Kapsayıcıyı kaldırır.
- **docker update [KAPSAYICI]:** Kapsayıcıyı günceller.
- **docker build [dosyayolu]:** Dockerfile dosyasından bir imaj oluşturur.
- **docker build -t [dosyayolu]:** Dockerfile dosyasından bir imaj oluşturur ve etiketler.

Docker Komutları

- **docker pull [IMAJ]:** Kayıtçıdan (imaj deposundan) imaj çek.
- **docker push [IMAJ]:** Kayıtçıya imaj gönder.
- **docker import [URL/FILE]:** tarball dosyasından imaj oluştur.
- **docker commit [CONTAINER] [YENI_IMAJ_ADI]:** Bir kapsayıcıdan imaj oluştur.
- **docker rmi [IMAJ]:** İmajı kaldır.
- **docker load [TAR_DOSYASI/STDIN_DOSYASI]:** tar arşivinden ya da el ile girilen dosya yolundan imaj yükle.
- **docker save [IMAJ] > [TAR_DOSYASI]:** Bir imaj dosyasını tüm katmanları, etiketleri, versiyonları ile tar arşiv dosyası olarak kaydet.
- **docker network ls:** Ağları listele.
- **docker network rm [AĞ]:** Bir ya da daha fazla ağı kaldır.
- **docker network inspect [AĞ]:** Bir ya da daha fazla ağ ile ilgili bilgileri göster.
- **docker network connect [AĞ] [KAPSAYICI]:** Kapsayıcıyı belirtilen ağa bağla.
- **docker network disconnect [AĞ] [KAPSAYICI]:** Kapsayıcıyı belirtilen ağdan çıkar.

Docker Komutları

- **docker start [KAPSAYICI]:** Kapsayıcıyı başlat.
- **docker stop [KAPSAYICI]:** Kapsayıcıyı durdur.
- **docker restart [KAPSAYICI]:** Kapsayıcıyı durdur ve yeniden başlat.
- **docker pause [KAPSAYICI]:** Çalışan kapsayıcıdaki işlemleri duraklat.
- **docker unpause [KAPSAYICI]:** Çalışan kapsayıcıdaki işlemleri çalıştırmaya devam et.
- **docker wait [KAPSAYICI]:** Diğer kapsayıcılar durana kadar kapsayıcıyı beklet.
- **docker kill [KAPSAYICI]:** Çalışan bir kapsayıcıyı SIGKILL sinyali göndererek kapat.
- **docker attach [KAPSAYICI]:** Çalışan bir kapsayıcıya standart giriş, çıkış ve hata akışlarını bağla.

Docker Komutları

- **docker ps:** Çalışan kapsayıcıları göster.
- **docker ps -a:** Çalışan ve durmuş kapsayıcıları göster.
- **docker logs [KAPSAYICI]:** Çalışan kapsayıcının loglarını göster.
- **docker inspect [NESNE_ADI/ID]:** Bir nesne hakkındaki alt düzey bilgileri göster.
- **docker events [KAPSAYICI]:** Kapsayıcıdaki gerçek zamanlı olayları göster.
- **docker port [KAPSAYICI]:** Kapsayıcıdaki port bilgilerini göster.
- **docker top [KAPSAYICI]:** Kapsayıcıdaki çalışan işlemleri göster.
- **docker stats [KAPSAYICI]:** Kapsayıcının anlık kaynak kullanım bilgilerini göster.
- **docker diff [KAPSAYICI]:** Dosya ya da dosya sistemindeki değişiklikleri göster.
- **docker images:** Yerel olarak depolanmış imajları göster.
- **docker history [IMAJ]:** İmajın tarihçesini göster.

Docker-Fullstack Uygulamasını Kapsülleme

- Front-End, REST API ve MongoDB'den oluşan civardaki mekanları gösteren, mekan eklemenizi, silmenizi, güncellemenizi sağlayan MekanBul isimi web tabanlı bir uygulamanız olduğunu düşünün.
- Her bir katmanı ayrı bir Docker kapsayıcısında (Toplam 3 adet) çalıştırmak istiyorsunuz.
- MongoDB kapsayıcınızda (mongodb_kapsayici) MongoDB 27017 portunda çalışacak.
- Front-End (mekanbul_kapsayici) kapsayıcınızda Front-End uygulamanız 3000 portunda çalışacak.
- REST API (mekanbuapi_kapsayici) kapsayıcınızda REST API'nız 9000 portunda çalışacak.
- Front-End REST API aracılığı ile MongoDB'ye bağlanacak.
- Front-End uygulama klasörümüz mekanbul_app ve bu klasörde bir Dockerfile tanımlayacağız.
- REST API klasörümüz mekanbul_api ve bu klasörde de Dockerfile tanımlayacağız.
- Birden fazla kapsayıcıyı ayarlamak ve dağıtmak için Docker içinde yer alan docker-compose aracını kullanacağınız.
- Docker-compose tüm servisleri tanımlamak için YAML adında bir dosya kullanır.
- Servislerin isimleri, servislerin bağımlılıkları, çalışacakları portlar, kullanacağı ağ bu dosya içinde tanımlanır.
- Uygulama klasöründe iken aşağıdaki komutlar çalıştırılarak servisler oluşturulur.
 - **docker-compose build**
- Tüm servisleri aynı anda çalıştırmak için aşağıdaki komut kullanılır.
 - **docker-compose up**
- REST API servisini kullanırken adres <http://servisadi:port> şeklinde koddan değiştirilmeli. servisadi ve port yml dosyasında tanımlanan ad ve porttur.
- Aynı şekilde MongoDB bağlantı adresiniz mongodb://mongodbservisadi:port/veritabaniadi şeklinde olmalı.

Docker-Fullstack Uygulamasını Kapsülleme

```
Dockerfile
1
2 FROM node:latest
3
4 WORKDIR /mekanbul_app
5
6 COPY package*.json ./
7
8 RUN npm install
9
10 COPY . .
11
12 EXPOSE 3000
```

Front-END Dockerfile

```
Dockerfile
1
2 FROM node:latest
3
4 WORKDIR /mekanbul_api
5
6 COPY package*.json ./
7
8 RUN npm install
9
10 COPY . .
11
12 EXPOSE 9000
```

Rest API Dockerfile

docker-compose.yml dosyası

```
1 version: '3.7'
2 services:
3   mekanbul_app:
4     build: .
5     image: asimsinanyuksel/mekanbul_app:server_app
6     container_name: mekanbul_kapsayici
7     ports:
8       - 3000:3000
9     networks:
10      - benimagim
11     restart: always
12     depends_on:
13       - mekanbul_api
14     command: npm start
15   mekanbul_api:
16     build: ./mekanbul_api
17     image: asimsinanyuksel/mekanbul_api:app_api
18     container_name: mekanbulapi_kapsayici
19     ports:
20       - 9000:9000
21     networks:
22      - benimagim
23     restart: always
24     depends_on:
25       - mongo
26     command: npm start
27   mongo:
28     image: mongo:latest
29     container_name: mongodb_kapsayici
30     networks:
31      - benimagim
32     restart: always
33     environment:
34       - MONGO_DATA_DIR=/data/db
35     volumes:
36       - ./data/db:/data/db
37     ports:
38       - 27017:27017
39 networks:
40   benimagim:
41     driver: bridge
42   ipam:
43     driver: default
```

yaml dosyasının sürümü

servisler bu anahtar kelime ile tanımlanır

derleme esnasında kullanılacak dosyaların konumu

oluşturulacak imajın adı

oluşturulacak kapsayıcının adı

servisin kullandığı portların listesi

servisin dahil olduğu ağ

bir sorun olduğunda yeniden başlat

servisin bağlı olduğu diğer servisler

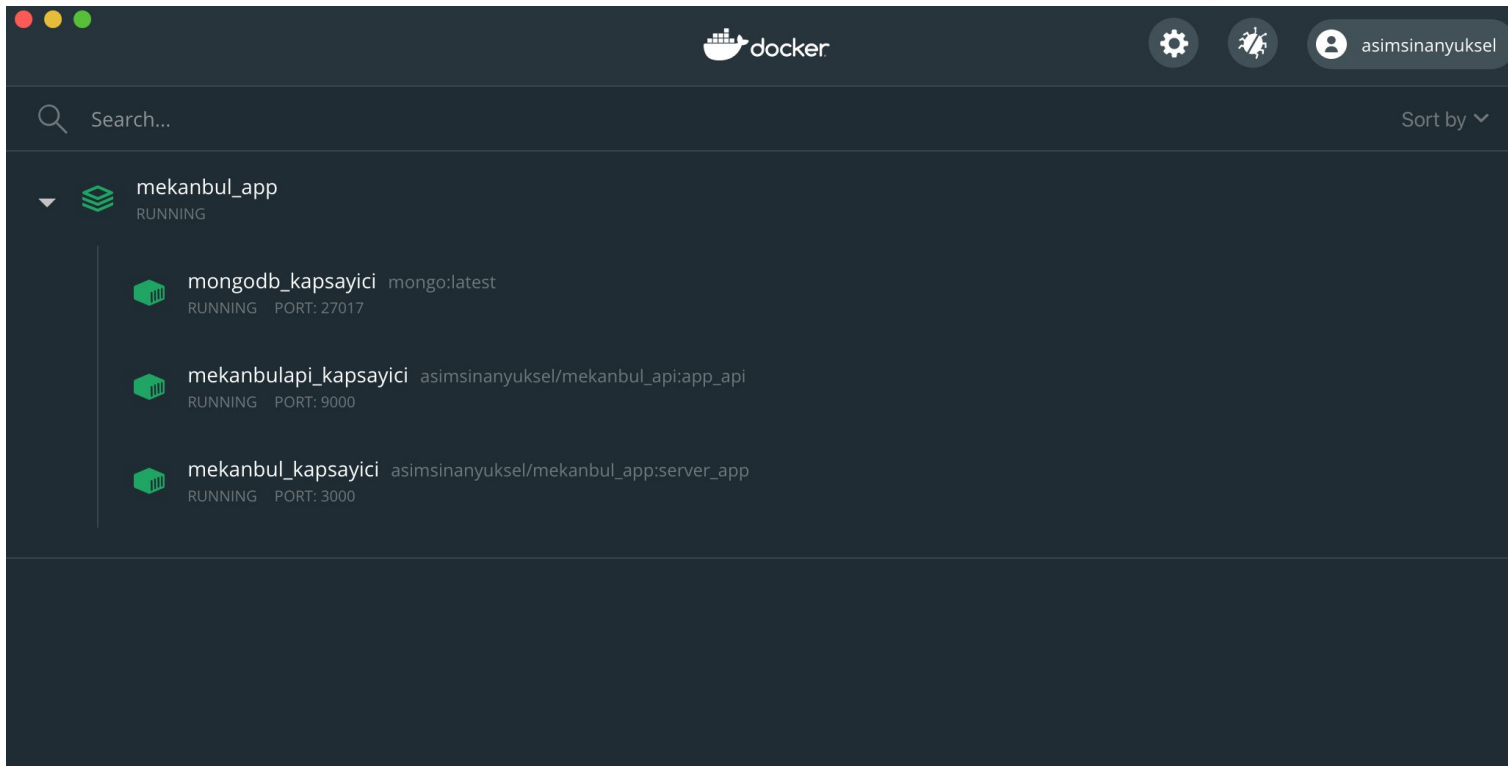
servis oluşturulduğunda çalışacak komut

servislerin kullandığı ağların tanımının yapıldığı yer

oluşturulan ağın adı

ağın türü

Docker for Desktop Dashboard



Servisleri durdurmak için: **docker-compose down**

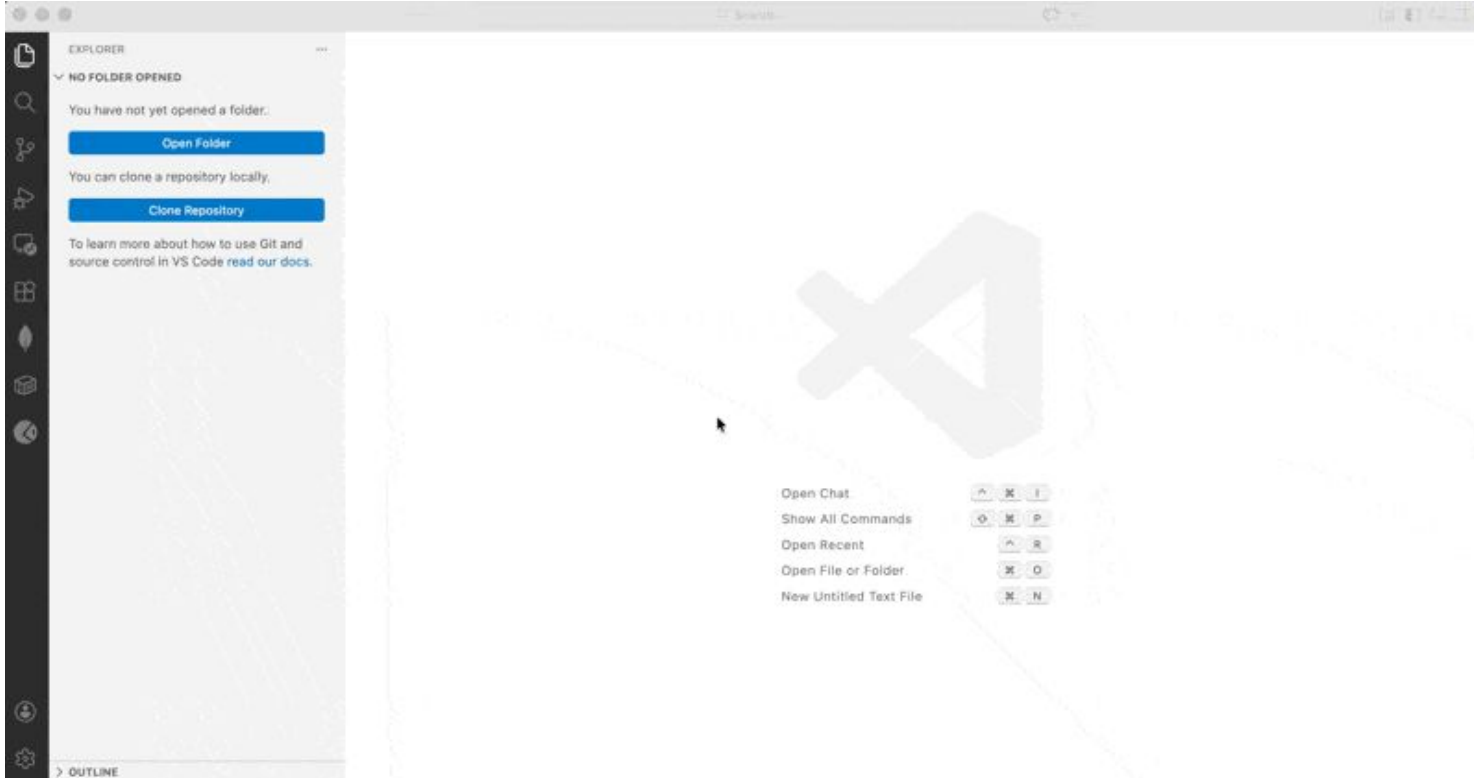
Servisleri DockerHub'a göndermek için: **docker-compose push**

Uygulama Github Adresi

Ek kaynaklar: <https://github.com/veggemonk/awesome-docker>

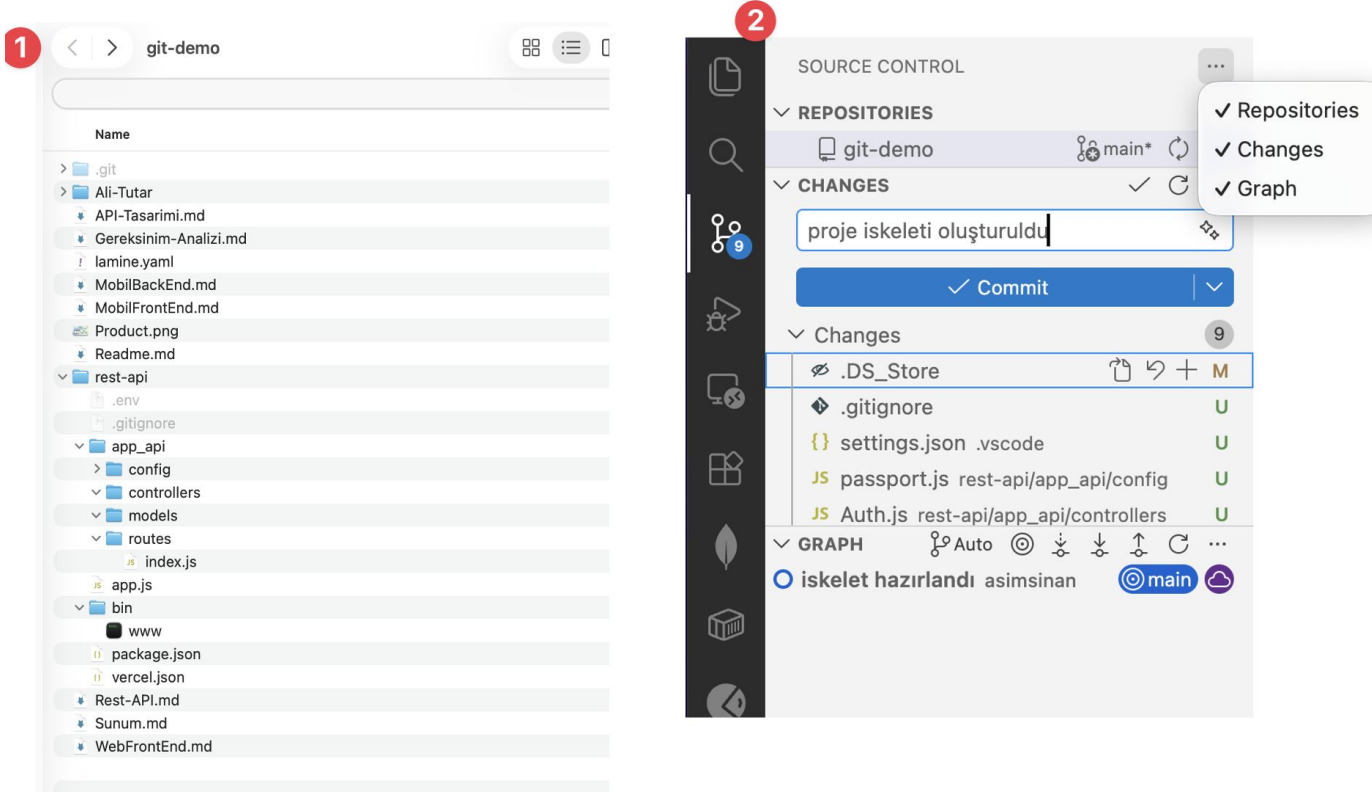
Git ile Takım Halinde Çalışma

- Repo adresinin <https://github.com/asimsinan/git-demo> olduğunu varsayalım.
- Proje repo sahibi grup üyesi ilk olarak repoyu kendi bilgisayarına kopyalamalı.
- Bunu iki şekilde yapabilirsiniz.
 - Komut satırından git clone <https://github.com/asimsinan/git-demo>
 - Ya da aşağıdaki gibi VSCode üzerinden



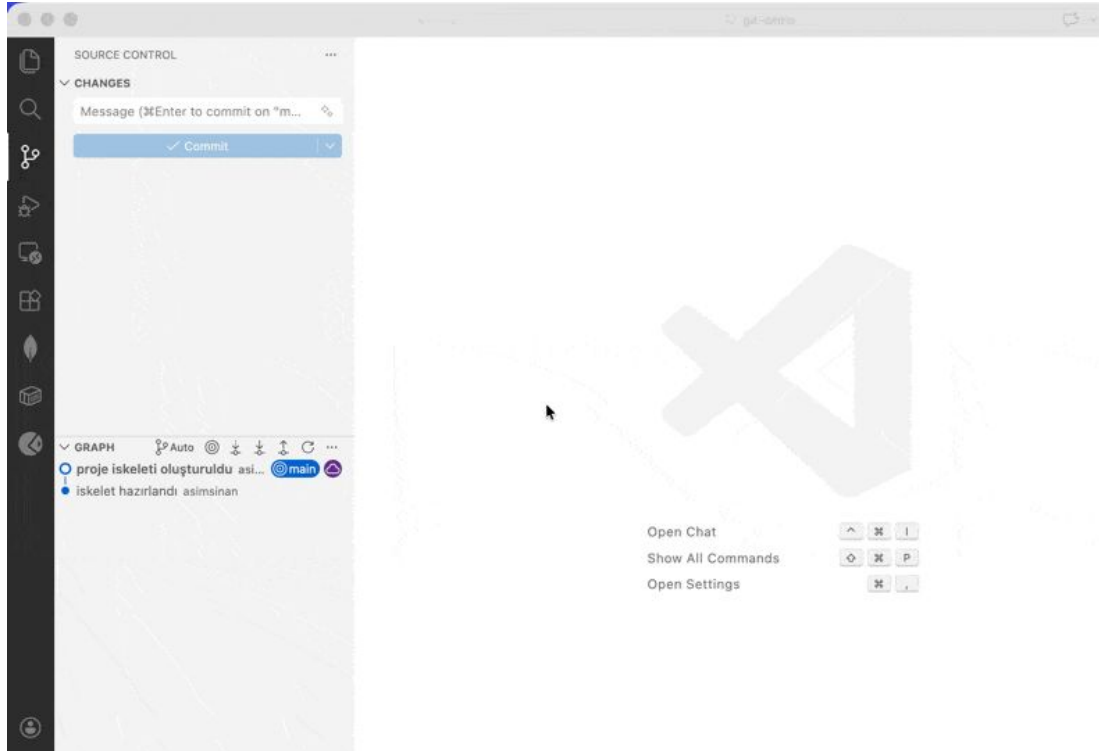
Git ile Takım Halinde Çalışma

- İkinci aşama olarak repo sahibi proje iskeletini bilgisayarında oluşturduktan sonra main branch'a commit ve push işlemlerini gerçekleştirmeli.
- Diyelim ki aşağıdaki gibi proje iskeletini oluşturdu.
 - Yaptığı değişiklikleri öncelikle aşağıdaki gibi commit daha sonra da push yapmalı



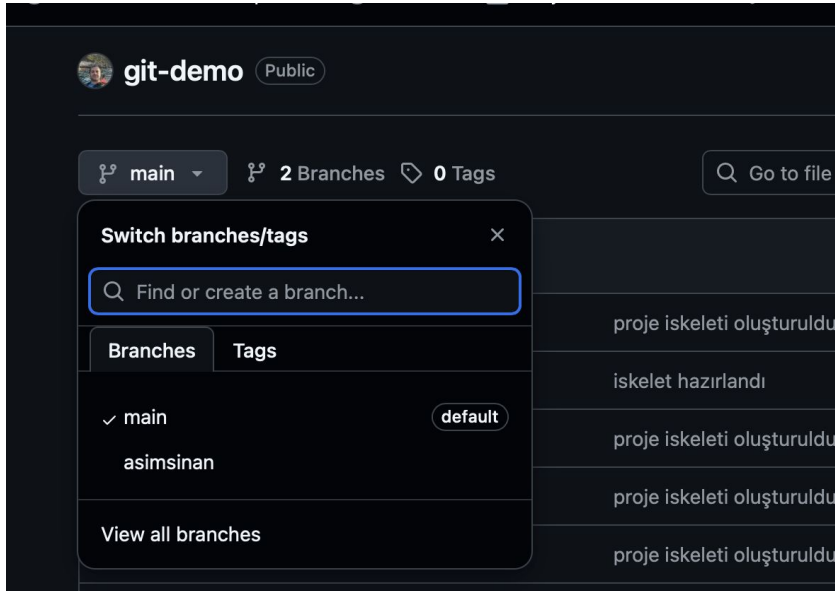
Git ile Takım Halinde Çalışma

- Bir sonraki aşamada her grup üyesinin bu iskeleti oluşturulan repoyu kendi bilgisayarlarına clone'lamaları gerekir. Bir önceki aşamada repoyu oluşturan kişinin bunu yapmasına gerek yok.
- Daha sonra herkesin kendi branch'ını oluşturma aşaması.
 - Herkes kendi adı ile (örnek benimki asimsinan olsun) aşağıdaki gibi **create branch, checkout to asimsinan, publish to branch** işlemlerini gerçekleştirmeli.



Git ile Takım Halinde Çalışma

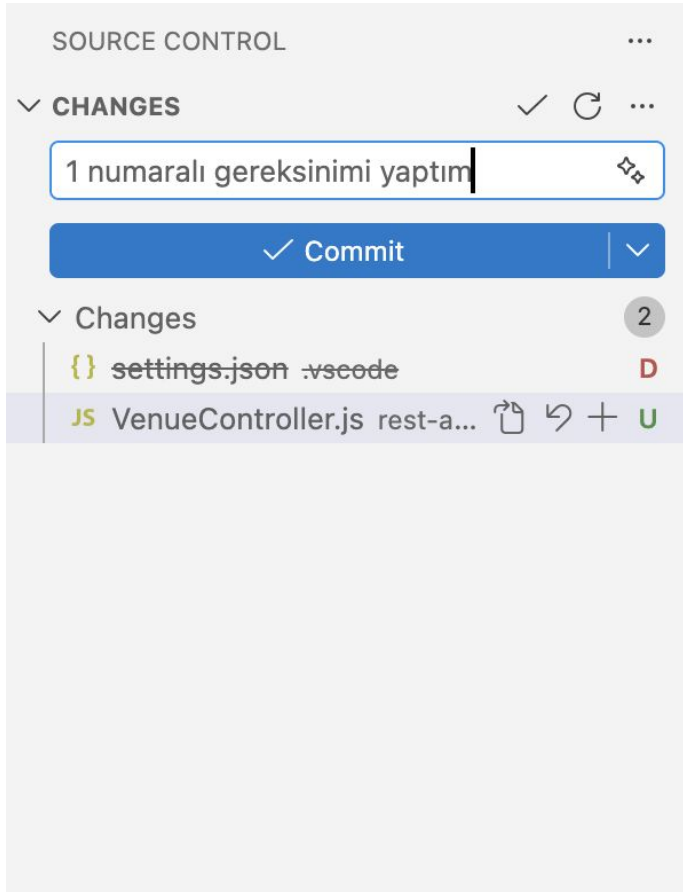
- Git reponuza gittiğinizde aşağıdaki gibi kendi branch'ınızın oluştuğunu göreceksiniz. Grup kaç kişiye o kadar branch olmalı.



- VSCode içinde de artık kendi branch'ınız içindesiniz. Yaptığınız değişiklikler sadece kendi branch'ınızı etkileyecek.
- Diyelimki grup üyesi kendi gereksinimlerinden birini ya da hepsini yaptı. VenuController isimli dosyayı oluşturdu.

Git ile Takım Halinde Çalışma

- İlk yapması gereken işlem kendi branch'ında bu değişiklikleri commit etmek ve daha sonra Sync Changes tuşuna basıp uzak reposuyla senkronize etmek.



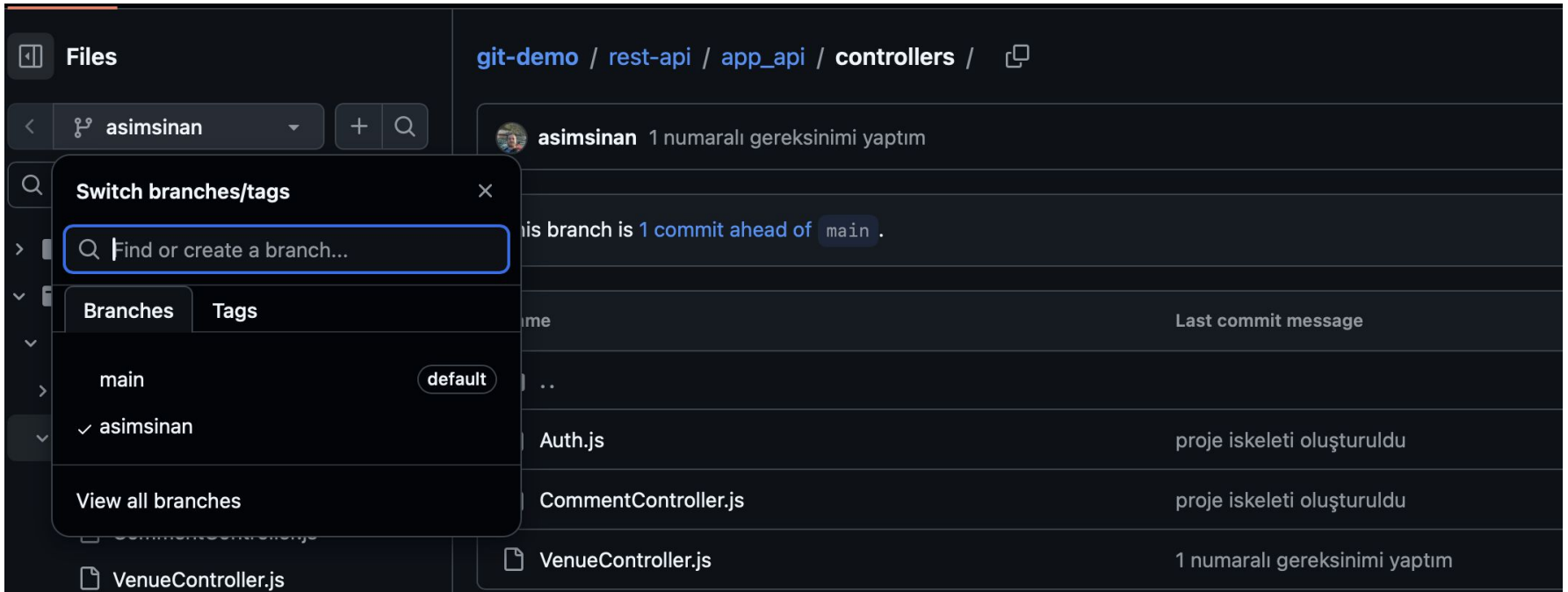
JS VenueController.js U X

rest-api > app_api > controllers > JS VenueController.js > ...

```
1 var mongoose = require("mongoose");
2 var Venue = mongoose.model("venue");
3 const createResponse = function (res, status, co
4   res.status(status).json(content);
5 };
6 var converter = (function () {
7   var earthRadius = 6371; // km
8   var radian2Kilometer = function (radian) {
9     return parseFloat(radian * earthRadius);
10  };
11  var kilometer2Radian = function (distance) {
12    return parseFloat(distance / earthRadius);
13  };
14  return {
15    radian2Kilometer,
```

Git ile Takım Halinde Çalışma

- Git reposuna gittiğinde asimsinan branch'ında değişikliklerin etkili olduğunu göreceksin. Ancak henüz main branch'ında hiçbir değişiklik yok (normal).



Git ile Takım Halinde Çalışma

- Şimdi diğer bir grup üyesinin kendi hesabından kendi gereksinimlerini yaptığını düşünelim. Unutmayın daha önceki aşamada " **create branch, checkout to asimsinan, publish to branch**" işlemlerini gerçekleştirilmiş olmalı.
- Diyelim ki bu grup üyesi CommentController.js'de değişiklikler yaptı. Commit ve sync işlemlerini gerçekleştirdi. Repoda yeni branch ve değişiklikler görünür olacaktır. **Ancak yapılanlar henüz main branch'a yansıtılmadı.**

SOURCE CONTROL

REPOSITORIES

git-demo... asinany* ✓ ↻ ...

CHANGES

3,4,5 nolu gereksinimler yapıldı.

✓ Commit

Changes 1

JS CommentController.js rest-api/a... M

AGENT REVIEW

Find Issues

Review diffs. ?

Files

asinany

Go to file

Ali-Tutar

rest-api

app_api

config

controllers

Auth.js

CommentController.js

VenueController.js

models

git-demo / rest-api / app_api / controllers /

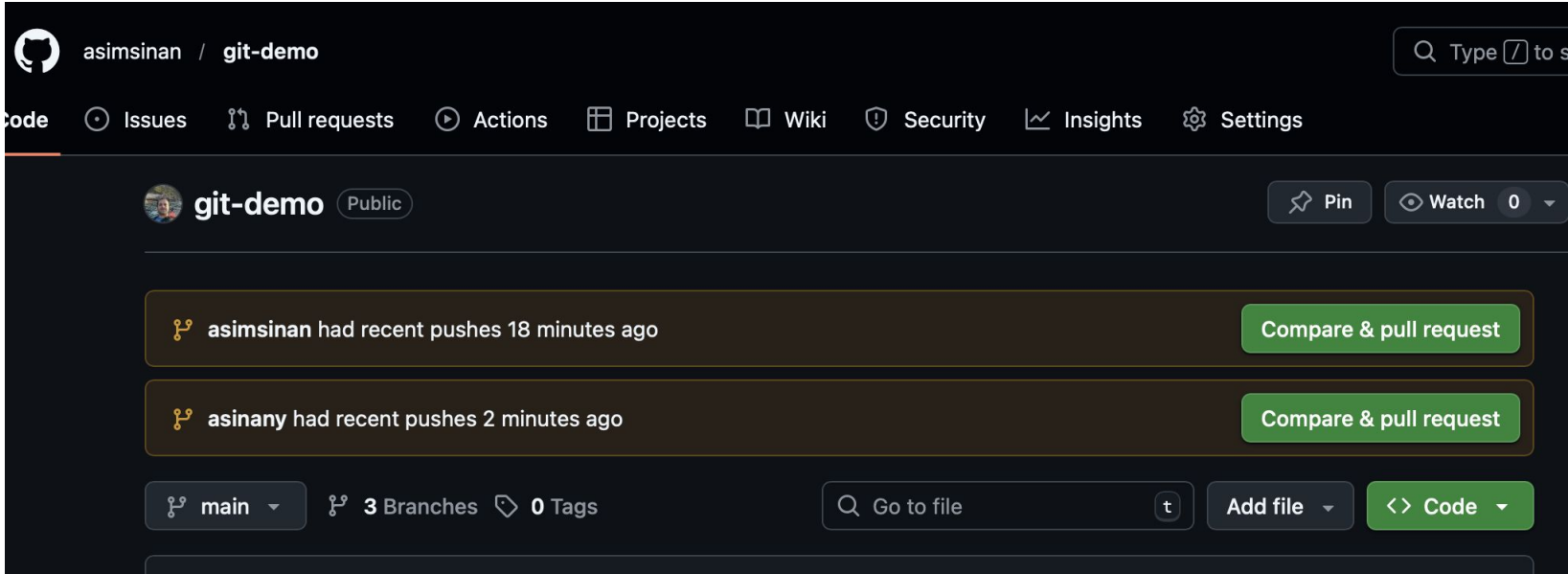
asimsinan 3,4,5 nolu gereksinimler yapıldı.

This branch is 2 commits ahead of main .

Name	Last commit message
..	
Auth.js	proje iskeleti oluşturuldu
CommentController.js	3,4,5 nolu gereksinimler yapıldı.
VenueController.js	1 numaralı gereksinimi yaptım

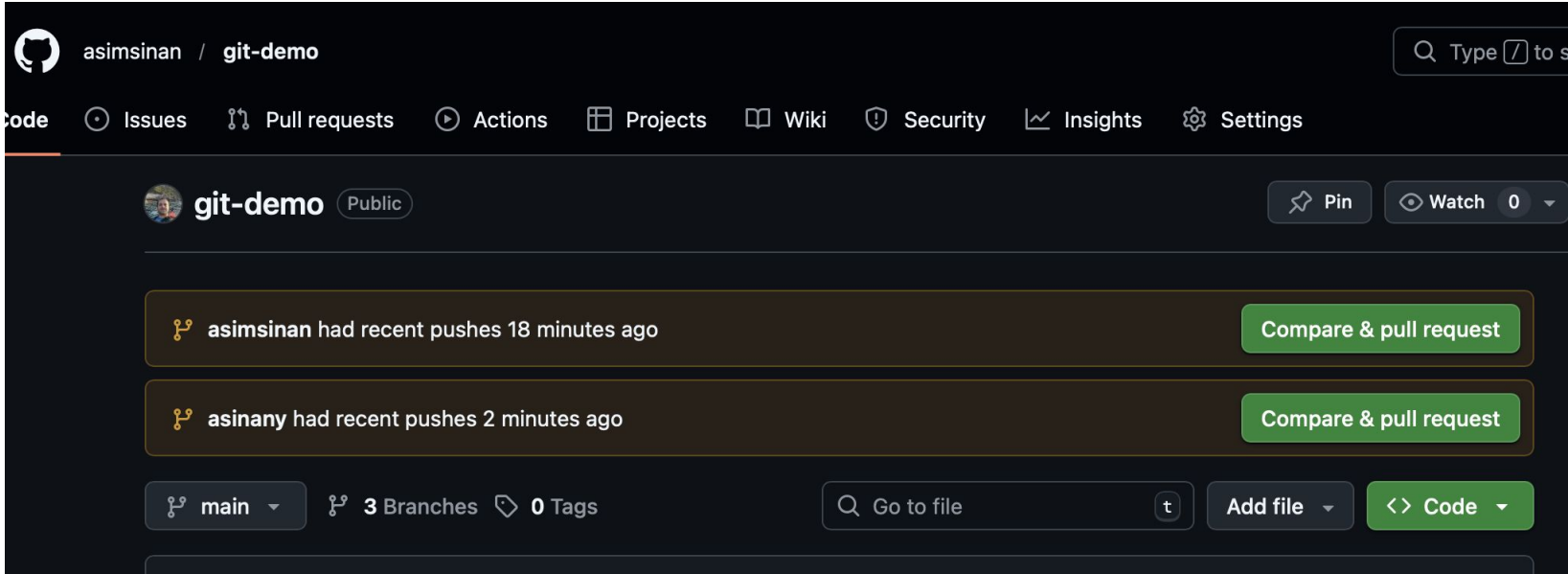
Git ile Takım Halinde Çalışma

- Github reposuna gittiğinizde aşağıdaki gibi bir uyarı ile karşılaşacaksınız.
- Bu farklı branch'larda yapılan değişikliklerin henüz main branch'a eklenmediği anlamına gelir.



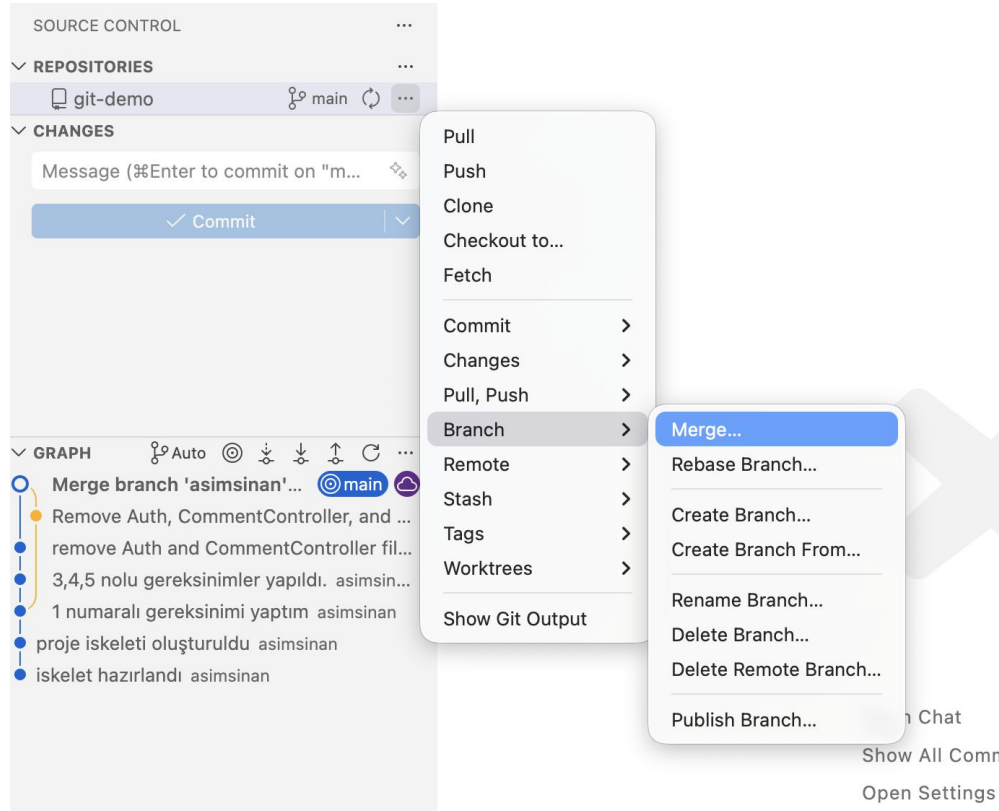
Git ile Takım Halinde Çalışma

- Github reposuna gittiğinizde aşağıdaki gibi bir uyarı ile karşılaşacaksınız.
- Bu farklı branch'larda yapılan değişikliklerin henüz main branch'a eklenmediği anlamına gelir.



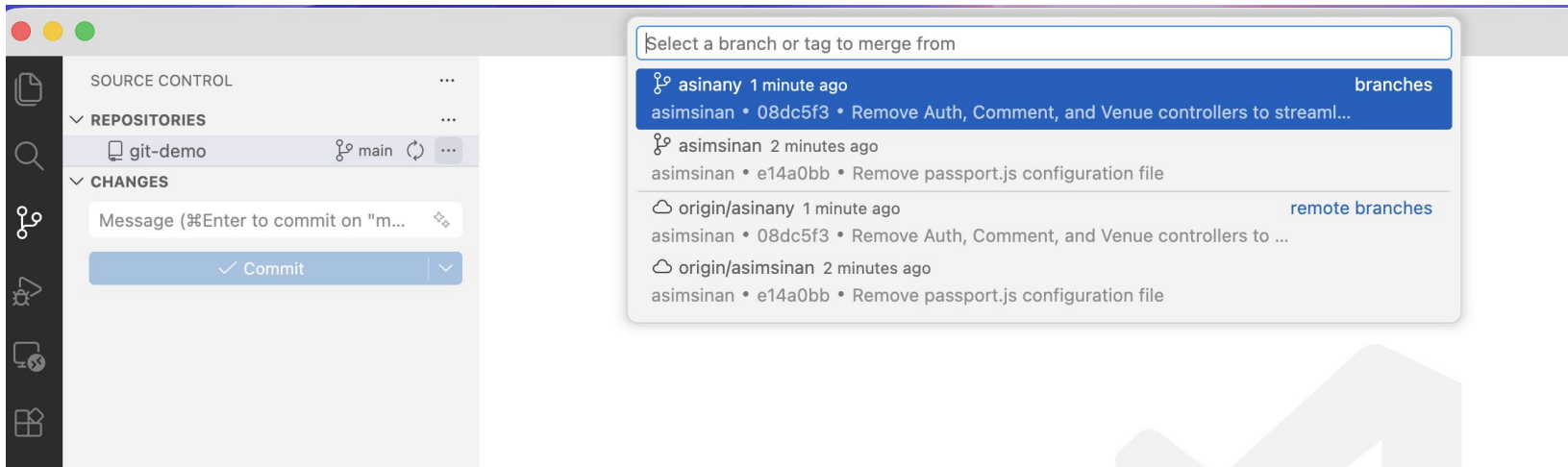
Git ile Takım Halinde Çalışma

- Tüm üyeler kendi branch'larında kendi gereksinimlerini yaptıktan ve değişikliklerini commit, sync yaptıktan sonra grup lideri tüm bu değişiklikleri main branch'a merge etmeli. Bu nedenle önce grup lideri **checkout to ...** diyerek main branch'a geçmeli. Daha sonra aşağıdaki gibi merge işlemini başlatmalı.



Git ile Takım Halinde Çalışma

- Merge olacak branch'lar sırayla seçilmeli ve merge işlemi gerçekleştirilmeli. Sync changes ile değişiklikleri kaydetmeli.
- Aşağıdaki örnekte 2 branch var. Dolayısıyla her bir için branch seçme, sync işlemi sırayla gerçekleştirilmeli.



- Eğer her grup üyesi kendine main branch güncellemelerini çekmek istiyorsa **checkout to main** diyerek main branch'a geçmeli ve **sync changes** diyerek değişiklikleri almalı.
- Eğer grup üyesi kendi branch'ına main branch'daki tüm değişiklikleri çekmek istiyorsa kendi branch'ında iken merge diyerek main branch'ı seçip merge yapabilir.
- Artık tüm grup üyelerinde main'in son hali olacak.

Ödev: 3. Ve 4. Aşama

- Derste anlattığım şekilde git üzerinden takım halinde çalışarak 3. ve 4. aşama için gerekli kodlar yazılacak.
- **Domain üzerinde çalışmayan ödevlere puan verilmez.**
- **Her grup üyesi kendi kullanıcı adıyla commit ve push yapmalı.**
- **Videonun başında isminizi, size ait gereksinim isimlerini ve bu gereksinimlerden kaç tanesi yaptığınızı belirtin. Örnek: 6 gereksinimin 4'ünü yaptım gibi.**
- **POSTMAN'de istek gönderdiğiniz domain adresi, koleksiyonunuz, koleksiyonunuzdaki API metotları, testin başarılı olduğuna dair kısımlar net bir şekilde görünmeli.**
- **Ödev teslim tarihi: 3 Nisan 2026 17:00**
- **MADDE 3. REST API:**
 - Api tasarımı yapılan gereksinimler grup üyeleri arasında eşit sayıda paylaştırılacak.
 - Api tasarımıdaki yol ve metotlara uygun bir şekilde her grup üyesi kendi üzerine düşen gereksinim için REST API metotlarını gerçekleştirecek ve kendi branch'ını oluşturacak commit ve push işlemlerini gerçekleştirecek.
 - REST API'nin yayınlandığı domain adresi (Örnek: restapi.vercel.com/v1 givbi) Rest API md dosyasının en başına yazılacak. Metotlar, yollar ve request body bilgileri de alta madde madde yazılacak
 - Grup üyeleri için gerekli REST API md sayfaları oluşturulacak. Her üye kendi sayfasını oluşturmalı.
 - Her grup üyesi POSTMAN üzerinden oluşturduğu koleksiyon üzerinden domain üzerindeki Rest API'sinin testini örnek verilerle yapacak ve başarılı olduğunu kanıtlayan bir youtube videosu çekerek kendi sayfasının en başına ekleyecek.
 - Her grup üyesi kendi gereksinimlerinin POSTMAN koleksiyonunu Json formatında dışarıya aktararak githubda daha önce kendi ismiyle oluşturduğu klasöre kaydedecek.

Ödev: 3. Ve 4. Aşama

- **Domain üzerinde çalışmayan ödevlere puan verilmez.**
- **Her grup üyesi kendi kullanıcı adıyla commit ve push yapmalı.**
- **Videonun başında isminizi, size ait gereksinim isimlerini ve bu gereksinimlerden kaç tanesi yaptığınızı belirtin. Örnek: 6 gereksinimin 4'ünü yaptım gibi.**
- **Her gereksinimin ismini test etmeden önce belirtilmeli ve testin başarılı olduğuna dair kanıtlar net bir şekilde gösterilmeli.**
- **Domain adresine girdiğimde site ulaşılabilir olmalı. Siteye ulaşamamam durumunda kanıtınız olsa bile puanı alamayacaksınız.**
- **Ödev teslim tarihi: 3 Nisan 2026 17:00**
- **MADDE 4. FRONT-END:**
 - Front end kısımları yapılacak gereksinimler grup üyeleri arasında eşit sayıda paylaştırılacak.
 - Üzerinize düşen REST API metotları yaptığınız frontende bağlanacaak ve kendi branch'inize commit ve push işlemlerini gerçekleştirecek.
 - Front-end'in yayınlandığı domain adresi (Örnek: front.vercel.com givbi) Web Frontend md dosyasının en başına yazılacak.
 - Grup üyeleri için gerekli frontend md sayfaları oluşturulacak. Her üye kendi sayfasını oluşturmalı.
 - Her grup üyesi proje domain adresi üzerinden frontendin gerekli API metotlarına bağlandığını ve doğru çalıştığını örnek verilerle test edecek ve veritabanı üzerinden işlemin başarılı olduğunu kanıtlayan bir youtube videosu çekerek kendi sayfasının en başına ekleyecek.
 - Her gereksinim için de ek detaylar video altında gösterilecek.